



CARE



CARE

HOSPITAL MANAGEMENT SYSTEM

Developer Guide

Care you Can Trust, Services You Deserve

PRESENTED TO
**APTECH E-PROJECT
DEPARTMENT**

PRESENTED BY
**FATIMA NAHID &
TEAM**

2407E1



Table of Contents

1

INTRODUCTION

2

ENVIRONMENT SETUP

3

PROJECT STRUCTURE

4

DATABASE DESIGN

5

DEVELOPMENT PHASES

6

CODING STANDARDS

7

GIT AND VERSION CONTROL

8

MODULE DESCRIPTIONS

9

TESTING & DEBUGGING

10

DEPLOYMENT

11

SECURITY PRACTICES

12

FUTURE IMPROVEMENTS

13

APPENDIX

14

CONTACT

1. *Introduction*



Welcome to the CARE Hospital Management System Developer Guide!

1.1 Overview:

This guide assists developers working on the CARE project, providing essential setup steps, best practices, workflows, and system knowledge. Whether you're joining the team or contributing as an open-source developer, this will streamline your onboarding and development.

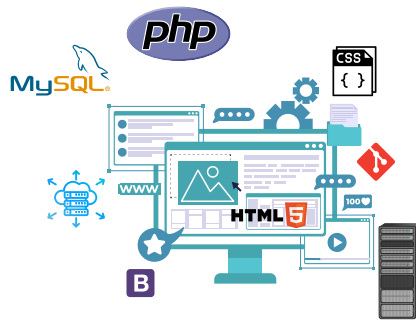
1.2 Purpose of this Document:

This guide serves developers working on CARE. It provides coding standards, structure references, version control practices, and deployment instructions for consistency and easy onboarding.

2. Environment Setup

2.1 TOOLS AND TECHNOLOGIES USED

- PHP 8+
- MySQL
- HTML5/CSS3
- Bootstrap 5
- XAMPP/WAMP
- Git/GitHub
- VS Code



2.2 INSTALLING REQUIRED SOFTWARE

- Install XAMPP/WAMP, a code editor like VS Code, and Git. Ensure Apache and MySQL are running.

2.3 SETTING UP THE PROJECT LOCALLY

- 1.Clone the repository: `git clone https://github.com/fatima-897/CARE.git`
- 2.Import hms.sql into phpMyAdmin from database folder
- 3.Place CARE directory in htdocs (XAMPP)
- 4.Visit `http://localhost/CARE/`

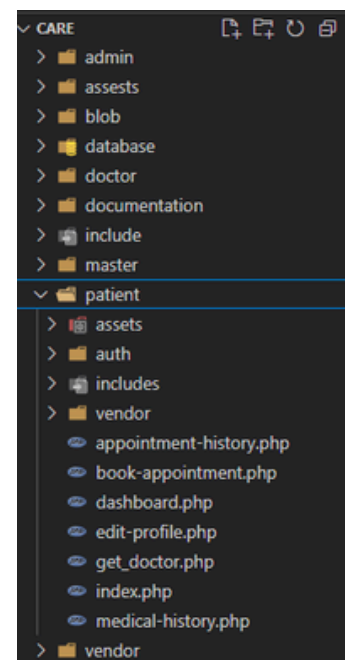
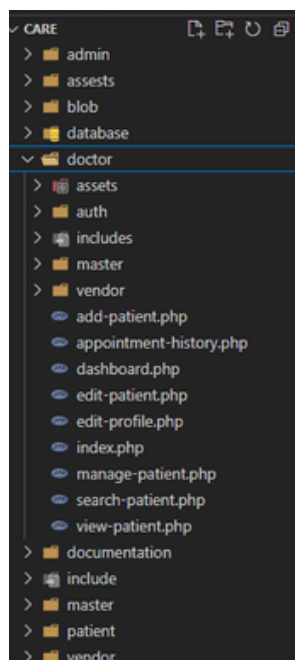
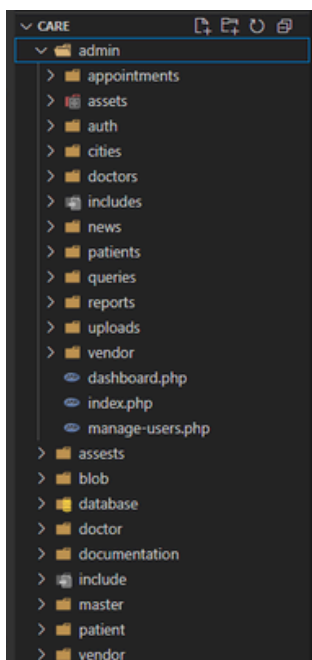
3. Project Structure

3.1 FOLDER OVERVIEW

admin/: Admin dashboard and functionality
assets/: CSS, JS, Images
blob/: readme screenshots
database/: hms.sql file
doctor/: Doctor dashboard and functionality
documentation/: CSS, JS, Images
include/: Config and session checks
master/: sass (styling purpose)
patient/: Patient dashboard and functionality
vendor/: mailing library
index.php/: home page
readme/: developers overview
LICENSE/: developed for educational purpose

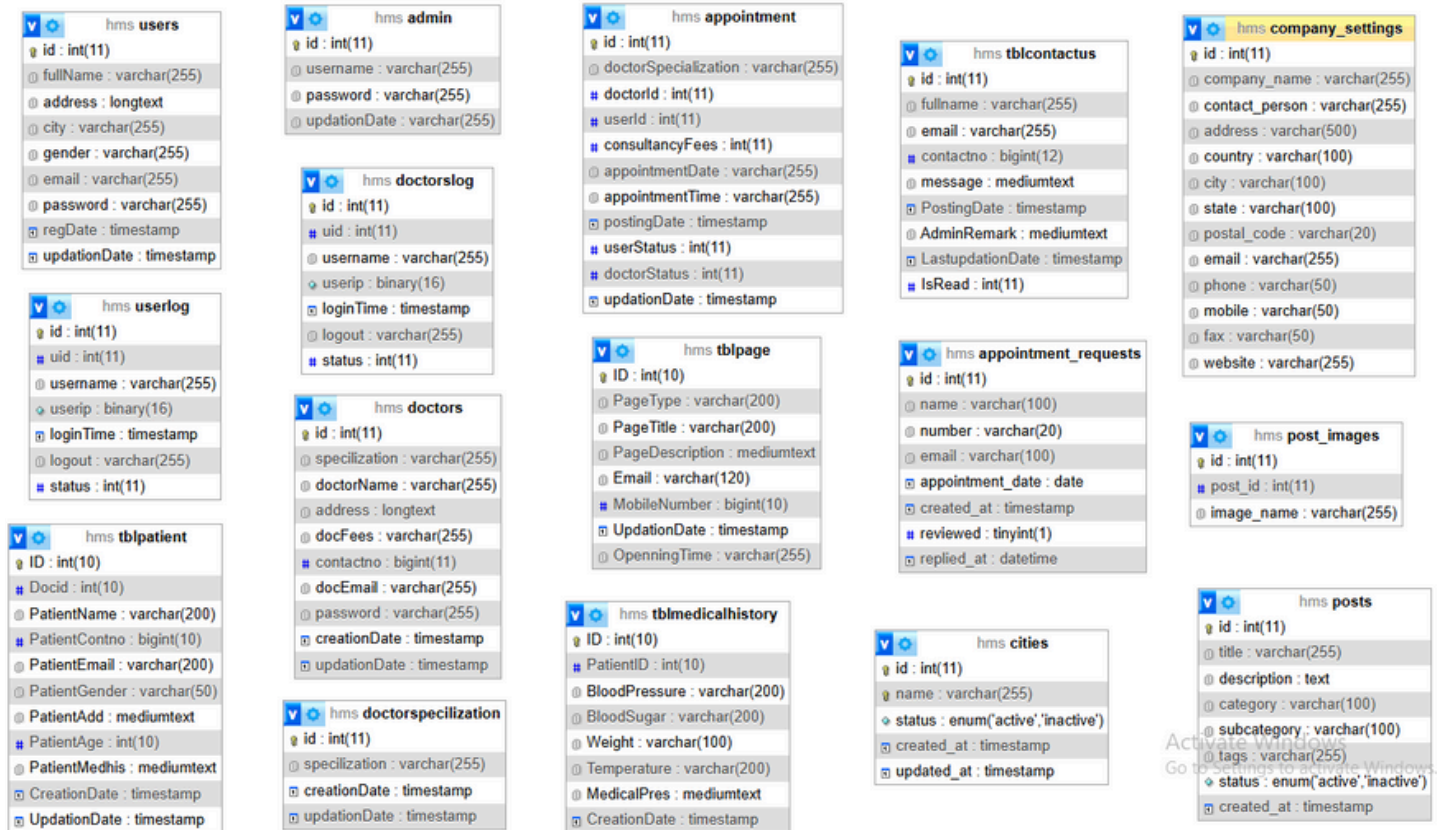
3.2 MODULE BREAKDOWN

- Each folder contains submodules like login, dashboard, CRUD operations, etc., separated by role.



4. Database Design

4.1 ENTITY-RELATIONSHIP DIAGRAM (ERD)



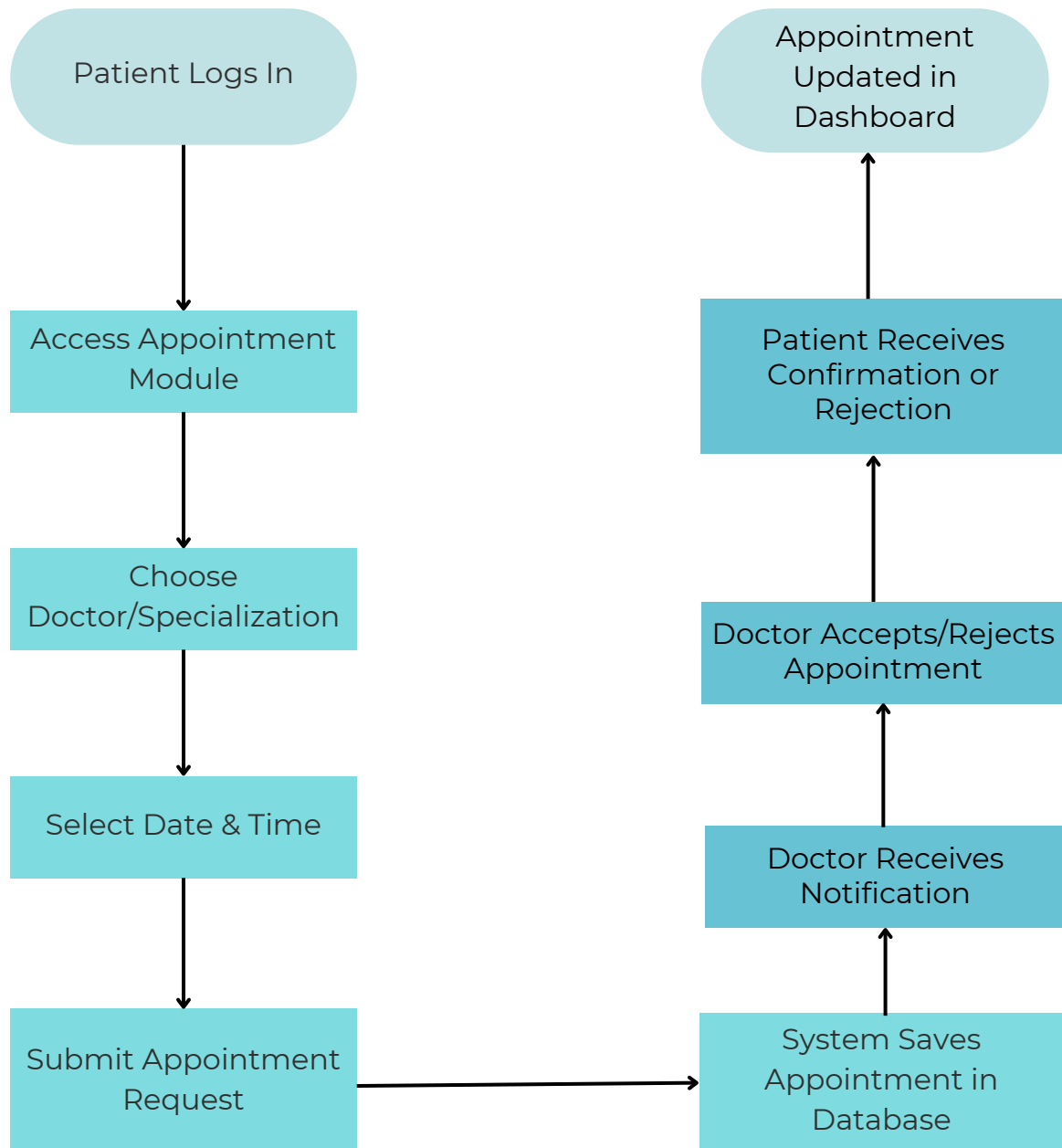
4. Database Design

4.2 TABLE DESCRIPTIONS

Table Name	Description	Primary Key (PK)	Foreign Key (FK)
users	Stores general user login information including role (admin/doctor/patient).	id	–
doctors	Stores doctor-specific profile and details.	id	user_id → users.id
patients	Stores patient profiles and contact information.	id	user_id → users.id
appointments	Manages scheduled appointments between patients and doctors.	id	patient_id → patients.id doctor_id → doctors.id
posts	Stores news and blog posts published by admin.	id	admin_id → users.id (admin role)
tblmedicalhistory	Contains medical records linked to patients and their appointments.	id	patient_id → patients.id doctor_id → doctors.id
contact_queries	Handles messages sent via the Contact Us page.	id	–

4. Database Design

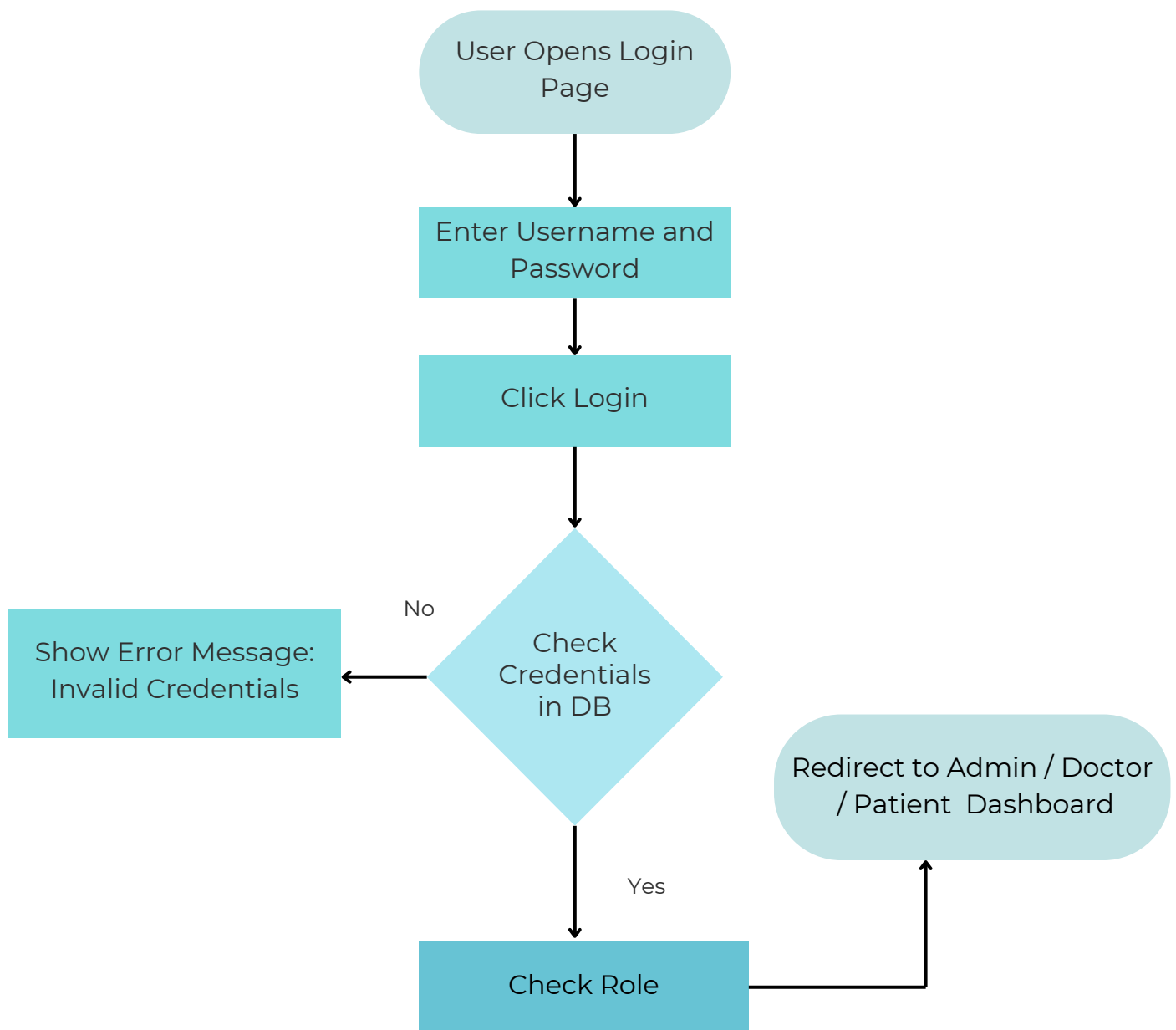
4.4 PROCESS FLOWCHARTS



Appointment Booking Flowchart

4. Database Design

4.4 PROCESS FLOWCHARTS



Login Process Flowchart (All Roles: Admin / Doctor / Patient)

5. Software Development Process

5.1 RESEARCH AND ANALYSIS

A thorough analysis was conducted to understand real-world hospital workflows and administrative needs. This included:

- Studying existing hospital management systems.
- Identifying common pain points in manual and semi-digital setups.
- Gathering functional requirements from healthcare professionals and staff.

The goal was to design a system that improves efficiency, ensures data accuracy, and streamlines doctor-patient interactions.

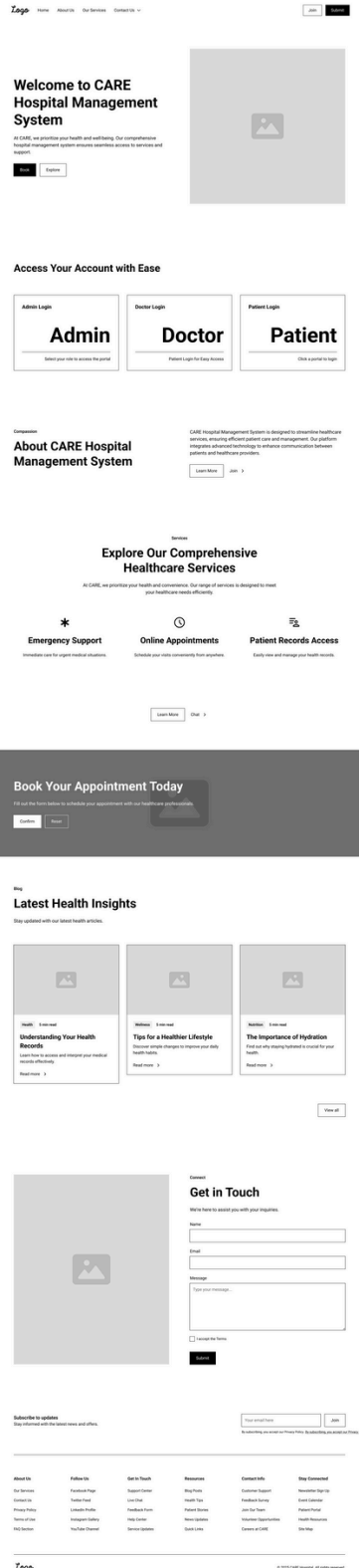
5.2 PLANNING AND STRATEGY

After establishing core requirements, we planned the system as follows:

- Scope: Focused on appointment scheduling, patient management, doctor dashboards, and admin control.
- User Roles: Clearly defined for Admin, Doctor, and Patient, with tailored dashboards.
- Technology Stack:
 - Frontend: HTML5, CSS3, Bootstrap
 - Backend: PHP
 - Database: MySQL
 - Server: Apache (XAMPP stack)
- Modules: Divided into folders and roles for clean separation:
 - admin/ for hospital staff and control
 - doctor/ for managing appointments and patients
 - patient/ for booking appointments and profile management

5.3 DESIGNING

- Wireframes and UI Mockups: Drafted using tools like Figma for each home page & dashboard.
- Database Schema: Normalized and relational design with tables like:
 - users, appointments, posts, tblmedicalhistory, doctors
- Flowcharts: Visual diagrams for:
 - Login Authentication Process
 - Appointment Booking System
 - Admin Module Navigation



5. Software Development Process

5.4 DEVELOPMENT

The core development phase involved building and integrating all major modules across user roles:

- Admin Panel: Appointment management, blog control, user records, and system settings.
- Doctor Panel: View/manage appointments, update patient histories, and access medical records.
- Patient Panel: Book appointments, view doctor availability, and manage profile details.

Security:

- Session-based route protection (checklogin.php)
- Input validation on both client and server side
- Controlled file uploads and error handling
- Modular Architecture: Code was organized into role-based folders (admin/, doctor/, patient/) with shared utilities under /includes.

```
1  <?php
2  function check_login()
3  {
4  if(strlen($_SESSION['login'])==
5  0)
6      {
7          $host = $_SERVER[
8  'HTTP_HOST'];
9          $uri = rtrim(dirname(
10 $_SERVER['PHP_SELF']), '/\');
11          $extra="./index.php"
12          ;
13          header(
14 "Location: http://$host$uri/
15 $extra");
16      }
17  }
```

5. Software Development Process

5.5 TESTING AND QUALITY ASSURANCE (QA)

Manual testing was carried out to ensure stability and functionality:

- **Functionality Testing:** Verified appointment flow, login/logout, form submissions, and CRUD operations.
- **UI Testing:** Checked cross-browser compatibility and responsive design for mobile/tablet.
- **Database Validation:** Ensured correct relational behavior, primary/foreign key enforcement, and data consistency.
- **Security Checks:**
 - Unauthorized access prevention
 - Form tampering safeguards
 - Session expiry and redirection handling
- **Bug Fixes:** Addressed minor layout issues, button visibility, and SQL error handling through iterative testing cycles.



6. Coding Standards

To ensure code readability, maintainability, and team collaboration, the following coding standards were adopted throughout the CARE Hospital Management System project:

6.1 FILE NAMING CONVENTIONS

- All file names are written in lowercase, using underscores to separate words.
- This convention ensures clarity, consistency, and compatibility across platforms.

Examples:

- `manage_doctors.php`
- `view_appointments.php`
- `edit_profile.php`

Folder Naming:

- Role-specific folders follow a simple, lowercase convention: `admin/`, `doctor/`, `patient/`, etc.

6.2 COMMENTING PRACTICES

- Inline comments (`//`) are used to explain logic, validation steps, and conditional branches.
- Block comments (`/* */`) are used for method headers, file/module descriptions, and complex code sections.
- Comments are written in clear and concise English, avoiding redundancy.
- Every major function or logic block begins with a brief comment header describing its purpose.

7. Git and Version Control

To maintain a clean, collaborative, and trackable development process, Git and GitHub were used for version control. The following conventions and practices were followed:

7.1 REPOSITORY STRUCTURE

- The codebase is hosted on GitHub for remote collaboration and history tracking.

7.2 COMMIT GUIDELINES

- Clear and consistent commit messages were used to describe changes accurately. This improves code traceability and team understanding.

Examples of Commit Messages:

- Added patient booking form with validation
- Fixed session timeout issue in doctor dashboard
- Refactored admin appointment controller for clarity
- Updated navbar to include logout for all roles

8. Module Descriptions

The CARE Hospital Management System is divided into modular components, each responsible for role-specific functionality. Below is a breakdown of each major module:

8.1 ADMIN MODULE

Purpose:

- Gives full control to hospital staff or administrators to manage core system data.

Features:

- Manage doctor records (add/edit/delete)
- Approve or cancel patient appointments
- View and manage registered patients
- Post, edit, or delete health blog/news articles
- Access dashboards and appointment statistics

Access Point:

- admin/index.php

```
1  <?php
2  session_start();
3  error_reporting(0);
4  include('includes/config.php');
5  if (isset($_POST['submit'])) {
6      $uname = $_POST['username'];
7      $upassword = $_POST['password'];
8
9      $ret = mysqli_query($con, "SELECT * FROM admin WHERE username='$uname' and password='$upassword'");
10     $num = mysqli_fetch_array($ret);
11     if ($num > 0) {
12         $_SESSION['login'] = $_POST['username'];
13         $_SESSION['id'] = $num['id'];
14         header("location:dashboard.php");
15     } else {
16         $_SESSION['errmsg'] = "Invalid username or password";
17     }
18 }
19 }
20 }
21 }
22
23 <!DOCTYPE html>
24 <html lang="en">
25
26 <head>
27     <meta charset="utf-8">
28     <title>Admin Login | CARE</title>
29     <meta name="viewport" content="width=device-width, initial-scale=1.0">
30     <link rel="shortcut icon" type="image/x-icon" href="assets/images/favicon.ico">
31
32     <!-- CSS -->
33     <link rel="stylesheet" href="assets/css/bootstrap.min.css">
34     <link rel="stylesheet" href="assets/css/font-awesome.min.css">
35     <link rel="stylesheet" href="assets/css/style.css">
36 </head>
37
38 <body>
39
40     <div class="main-wrapper account-wrapper">
41         <div class="account-page">
42             <div class="account-center">
43                 <div class="account-box">
44                     <form class="form-signin" method="post">
45                         <div class="account-logo">
46                             <a href="index.php"></a>
47                         </div>
48                         <h4 class="text-center">Admin Login</h4>
49
50                         <p style="color:red;" class="text-center">
51                             <php echo htmlentities($_SESSION['errmsg']); ?>
52                             <php echo htmlentities($_SESSION['errmsg']); ?>
53                         </p>
54
55                         <div class="form-group">
56                             <label>Username</label>
57                             <input name="username" type="text" class="form-control" placeholder="Enter username" required>
58                         </div>
59
60                         <div class="form-group">
61                             <label>Password</label>
62                             <input name="password" type="password" class="form-control" placeholder="Enter password" required>
63                         </div>
64
65                         <div class="form-group text-right">
66                             <a href="#">Forgot your password?</a>
67                         </div>
68
69                         <div class="form-group text-center">
70                             <button type="submit" name="submit" class="btn btn-primary account-btn">Login</button>
71                         </div>
72
73                         <div class="text-center register-link">
74                             <a href="index.php">Back to Home Page</a>
75                         </div>
76                     </form>
77                 </div>
78             </div>
79         </div>
80     </div>
81
82 <!-- JS -->
83 <script src="assets/js/jquery-3.6.0.min.js"></script>
84 <script src="assets/js/bootstrap.bundle.min.js"></script>
85 <script src="assets/js/login.js"></script>
86
87 <script>
88     jQuery(document).ready(function () {
89         Main.init();
90         Login.init();
91     });
92 </script>
93
94 </body>
95 </html>
```

8. Module Descriptions

8.2 DOCTOR MODULE

Purpose:

- Allows doctors to view their scheduled appointments and manage medical records.

Features:

- View upcoming appointments and assigned patients
- Add or update patient diagnosis and treatment history
- Access past medical records
- Manage personal profile settings

Access Point:

- doctor/index.php

```
1 <?php
2 session_start();
3 error_reporting(0);
4 include("includes/config.php");
5
6 if (isset($_POST['submit'])) {
7     $uname = $_POST['username'];
8     $dpasword = md5($_POST['password']);
9     $ret = mysqli_query($con, "SELECT * FROM doctors WHERE docEmail='$uname' and password='$dpasword'");
10    $num = mysqli_fetch_array($ret);
11    if ($num > 0) {
12        $_SESSION['dlogin'] = $_POST['username'];
13        $_SESSION['id'] = $num['id'];
14        $uid = $num['id'];
15        $uip = $_SERVER['REMOTE_ADDR'];
16        $status = 1;
17        //Code Logs
18        $log = mysqli_query($con, "insert into doctorslog(uid,username,userip,status) values('$uid','$uname','$uip','$status')");
19
20        header("location:dashboard.php");
21    } else {
22
23        $uip = $_SERVER['REMOTE_ADDR'];
24        $status = 0;
25        mysqli_query($con, "insert into doctorslog(username,userip,status) values('$uname','$uip','$status')");
26        echo "<script>alert('Invalid username or password');</script>";
27        echo "<script>window.location.href='index.php'</script>";
28    }
29 }
30 >
31
32 <!DOCTYPE html>
33 <html lang="en">
34
35 <head>
36 <meta charset="utf-8">
37 <title>Doctor Login | CARE</title>
38 <meta name="viewport" content="width=device-width, initial-scale=1.0">
39 <link rel="shortcut icon" type="image/x-icon" href="assets/images/favicon.ico">
40
41 <!-- CSS -->
42 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
43 <link rel="stylesheet" href="assets/css/bootstrap.min.css">
44 <link rel="stylesheet" href="assets/css/font-awesome.min.css">
45 <link rel="stylesheet" href="assets/css/style.css">
46 </head>
47
48 <body>
49
50 <div class="main-wrapper account-wrapper">
51 <div class="account-page">
52 <div class="account-center">
53 <div class="account-box">
54 <form class="form-signin" method="post">
55 <div class="account-logo">
56 <a href=".." index.php"></a>
57 </div>
58
59 <h4 class="text-center">Doctor Login</h4>
60
61 <p style="color:red;" class="text-center">
62 <?php echo htmlentities($_SESSION['errmsg']); >
63 <?php echo htmlentities($_SESSION['errmsg']) = ""; >
64 </p>
65
66 <div class="form-group">
67 <label>Email</label>
68 <input name="username" type="text" class="form-control" placeholder="Enter Email"
69 required>
70 </div>
71
72 <div class="form-group">
73 <label>Password</label>
74 <input name="password" type="password" class="form-control" placeholder="Enter password"
75 required>
76 </div>
77
78 <div class="form-group text-center">
79 <button type="submit" name="submit" class="btn btn-primary account-btn">Login</button>
80 </div>
81
82 <div class="text-center register-link">
83 <a href=".." index.php">Back to Home Page</a>
84 </div>
85 </form>
86 </div>
87 </div>
88 </div>
89 </div>
90
91 <!-- JS -->
92 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
93 <script src="assets/js/jquery-3.6.0.min.js"></script>
94 <script src="assets/js/bootstrap.bundle.min.js"></script>
95 <script src="assets/js/login.js"></script>
96 </script>
97
98 <?php
99 <script>
100 <script>
101 </script>
102 </body>
103
104 </html>
```

8. Module Descriptions

8.3 PATIENT MODULE

Purpose:

- Enables patients to interact with the system, book consultations, and manage their information.

Features:

- Register/login to the portal
- Book new appointments with available doctors
- View appointment history and status
- Edit personal profile details

Access Point:

- index.php (root of the application)

```
1 <?php
2 session_start();
3 error_reporting(0);
4 include("includes/config.php");
5 if (isset($_POST['submit'])) {
6     $username = $_POST['email'];
7     $password = md5($_POST['password']);
8
9     $ret = mysql_query($con, "SELECT * FROM users WHERE email='$username' and password='$password'");
10    $num = mysql_fetch_array($ret);
11
12    if ($num > 0) {
13        $_SESSION['login'] = $_POST['email'];
14        $_SESSION['id'] = $num['id'];
15        $uid = $num['id'];
16        $host = $_SERVER['HTTP_HOST'];
17        $uip = $_SERVER['REMOTE_ADDR'];
18        $status = 1;
19
20        // Successful login log
21        $log = mysql_query($con, "insert into userlog(uid,username,userip,status) values('$uid','$username','$uip','$status')");
22        header("location:dashboard.php");
23    } else {
24        // Failed login log
25        $_SESSION['login'] = $_POST['email'];
26        $uid = $_SERVER['REMOTE_ADDR'];
27        $status = 0;
28        mysql_query($con, "insert into userlog(uid,username,userip,status) values('$username','$uip','$status')");
29
30        echo "<script>alert('Invalid username or password');</script>";
31        echo "<script>window.location.href='user-login.php';</script>";
32    }
33 }
34 //
35 <!DOCTYPE html>
36 <html lang="en">
37 <head>
38 <meta charset="UTF-8">
39 <meta name="viewport" content="width=device-width, initial-scale=1.0">
40 <link rel="shortcut icon" href="assets/images/favicon.ico">
41 <title>User Registration | Patient</title>
42
43 <!-- CSS -->
44 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
45 <link rel="stylesheet" href="assets/css/bootstrap.min.css">
46 <link rel="stylesheet" href="assets/css/font-awesome.min.css">
47 <link rel="stylesheet" href="assets/css/style.css">
48
49 </head>
50
51 <body>
52 <div class="main-wrapper account-wrapper py-5 bg-light">
53 <div class="account-page">
54 <div class="account-center">
55 <div class="account-box shadow">
56 <form class="form-signin p-4" method="post">
57 <div class="account-logo text-center mb-4">
58 <a href="index.php" img src="assets/images/logo-dark.png" alt="CARE Logo" class="img-fluid" style="height: 60px;"></a>
59 </div>
60 <h4 class="text-center mb-4">Patient Login</h4>
61 <p class="text-center text-danger">
62 <php echo htmlentities($_SESSION['errmsg']); >
63 <php echo htmlentities($_SESSION['errmsg']) . "</p>"; >
64
65 <div class="form-group mb-3">
66 <label for="email">Email</label>
67 <input type="email" class="form-control" name="email" id="email" placeholder="Email" required>
68 </div>
69
70 <div class="form-group mb-3">
71 <label for="password">Password</label>
72 <input type="password" class="form-control" name="password" id="password" placeholder="Enter password" required>
73 </div>
74
75 <div class="form-group text-start mb-3">
76 Don't have an account? <a href="auth/register.php">Create an account</a>
77 </div>
78
79 <div class="d-grid mb-3">
80 <button type="submit" name="submit" class="btn btn-primary">Login</button>
81 </div>
82
83 <div class="text-center">
84 <a href="index.php">Back to Home Page</a>
85 </div>
86 </form>
87 </div>
88 </div>
89 </div>
90
91 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
92 <script src="assets/js/jquery-3.6.0.min.js"></script>
93 <script src="assets/js/bootstrap.bundle.min.js"></script>
94 <script src="assets/js/login.js"></script>
95
96 <script>
97 $(document).ready(function() {
98     Main.init();
99     Login.init();
100 });
101 </script>
102 </body>
103 </html>
```

8. Module Descriptions

8.4 INCLUDES AND ASSETS

Includes /includes/

- config.php: Database connection configuration
- checklogin.php: Session/login authentication logic
- header.php, footer.php: Shared layout templates for consistent UI

Assets /assets/

- CSS files for design and styling
- Images, icons, and favicon
- JavaScript files for frontend interactivity

These components ensure code reusability, maintainability, and consistent styling across modules.

```
1 <?php
2 define('DB_SERVER', 'localhost');
3 define('DB_USER', 'root');
4 define('DB_PASS', '');
5 define('DB_NAME', 'hms');
6
7 $con = mysqli_connect(DB_SERVER, DB_USER, DB_PASS, DB_NAME);
8
9 // Check connection
10 if (mysqli_connect_errno()) {
11     echo "Failed to connect to MySQL: " . mysqli_connect_error();
12     exit();
13 }
```

9. Testing & Debugging

9.1 MANUAL TESTING

Test Cases Covered:

- **Login/Logout:** Verified correct redirection and session creation for admin, doctor, and patient roles.
- **Appointment Flow:** Checked if patients can book, update, and cancel appointments and whether doctors/admins see these changes.
- **Form Submissions:** Ensured proper validation messages and required fields across all forms.

CRUD Operations:

 Manually tested creation, editing, and deletion of:

- Doctors (by Admin)
 - Posts (by Admin)
 - Patient history (by Doctor)
-
- **UI/UX:** Verified responsive design and page alignment across browsers (Chrome, Firefox).

9.2 TROUBLESHOOTING COMMON ERRORS

Issue	Possible Cause	Fix
Login fails	<code>\$_SESSION['login']</code> is not set properly	Confirm <code>session_start()</code> is called and credentials match in DB
Form not submitting	HTML name attributes missing or JavaScript validation blocking	Check field names, required attributes, and browser console
Database not updating	Query returns error	Inspect SQL syntax, debug with <code>mysqli_error()</code>
Broken images or CSS	Incorrect path or file not found	Check folder structure and use relative paths
Session not maintained after login	Session not initialized or misconfigured	Ensure <code>session_start()</code> is at the top of each page using sessions

9. Testing & Debugging

Debugging Tools Used:

- `var_dump()` and `print_r()` for variable inspection
- Browser Developer Tools (F12) for JS and network request analysis
- phpMyAdmin to monitor database changes and test SQL queries directly

10. Deployment

10.1 PREPARING FOR PRODUCTION

Before deploying the system, several steps were taken to ensure security and reliability:

Input Sanitization

- All user inputs are validated and sanitized using PHP functions like `htmlspecialchars()`, `mysqli_real_escape_string()`, and server-side checks to prevent SQL injection and XSS attacks.

Error Handling

- In production, error reporting is suppressed and logs are stored securely:



```
1 error_reporting(0);
2 ini_set('display_errors', 0);
```

Security via .htaccess

- Prevent access to sensitive files (e.g., `config.php`)
- Force HTTPS (if SSL is enabled)
- Disable directory browsing

10. Deployment

10.2 HOSTING RECOMMENDATIONS

To run the system smoothly, the following hosting environment is recommended:

- Server: Apache (LAMP stack)
- PHP Version: 8.0 or higher
- Database: MySQL (InnoDB engine)

Other Requirements:

- SMTP enabled (for email notifications if integrated)
- File upload permissions (for blog images)

Recommended options include:

- Shared hosting providers like Hostinger, Bluehost, or Namecheap
- Local deployment via XAMPP for development/testing

11. Security Practices

11.1 INPUT VALIDATION

Proper validation and sanitization were applied to all user-submitted data to prevent attacks such as SQL injection, XSS, and malformed input.

Techniques Used:

- htmlspecialchars() to neutralize special characters in HTML output.
- Regex-based validation for email, phone numbers, and usernames.
- is_numeric(), filter_var(), and type checking to ensure data integrity.
- Required field checks on both client (JavaScript) and server sides.

```
$name = htmlspecialchars($_POST['name']);
```

```
$email = filter_var($_POST['email'], FILTER_VALIDATE_EMAIL);
```

11.2 SESSION MANAGEMENT

Session handling ensures that only authenticated users access protected resources.

Practices Implemented:

- session_start() included on all pages using session data.
- Session variables checked on each protected route using checklogin.php.
- session_destroy() is called during logout to fully clear user session.
- Unauthorized access attempts are redirected to the login page.

```
if (strlen($_SESSION['login']) == 0) {  
    header("Location: ./index.php");  
    exit();  
}
```

11. Security Practices

11.3 PASSWORD AND DATA PROTECTION

User credentials and sensitive data are handled with care to avoid exposure.

Measures Taken:

- Passwords are stored using md5() hashing (Note: MD5 is not recommended for production).
- Data queries are validated and escaped using mysqli_real_escape_string()

For example:

```
$password = md5($_POST['password']);
```

12. *Future Improvements*

While the current CARE Hospital Management System covers essential hospital operations, there are several enhancements planned to further improve functionality, user engagement, and integration with modern health services.

12.1 FEATURE ROADMAP

Future versions of the system aim to include the following user-focused upgrades:

Mobile App Integration

- Develop a companion mobile app (Android/iOS) for patients and doctors to:
 - Book or manage appointments on the go
 - Get real-time notifications
 - Access health records instantly

Patient Notification System

- Implement email/SMS alerts for:
 - Appointment confirmations/reminders
 - Prescription updates
 - Health blog/news publication

Doctor Availability Calendar

- Interactive calendar for doctors to:
 - Set custom availability slots
 - Automatically block booked timeframes
 - Sync with patient appointment requests

12.2 POTENTIAL INTEGRATIONS

To extend system capabilities and modernize healthcare workflows, the following third-party integrations are under consideration:

SMS Gateway API

- Integrate with services like Twilio, Nexmo, or Pakistani SMS providers to send real-time appointment updates and OTPs.

Diagnostics and Referral APIs

- Connect with labs or partner hospitals to:
 - Retrieve diagnostic reports directly
 - Auto-generate referrals for specialist cases
 - Upload and store lab results in patient history

13. Appendix

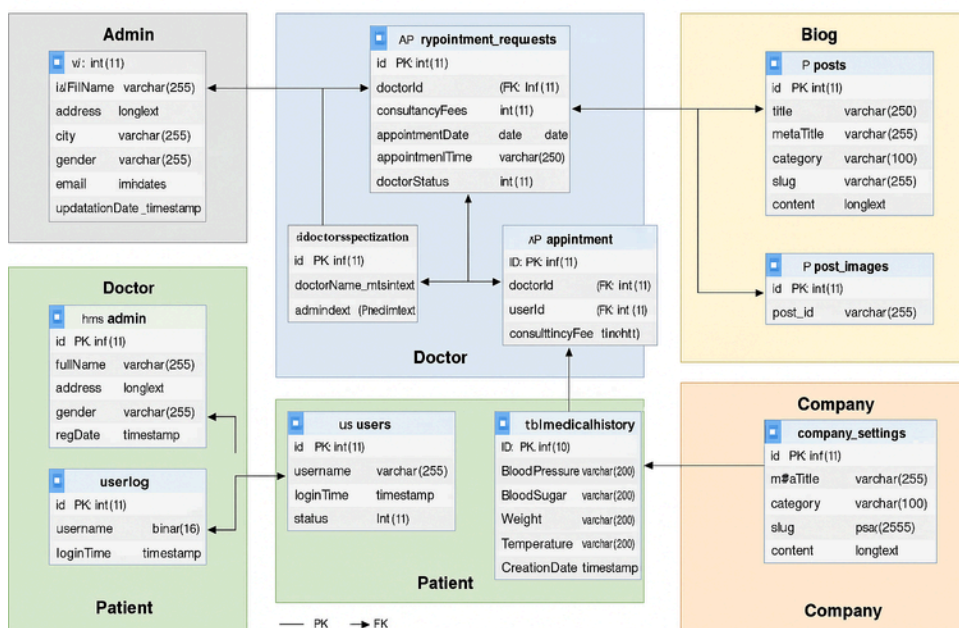
This section provides supporting materials for setup and understanding of the system's backend structure, including configuration files and database overview.

13.1 SQL FILE OVERVIEW

The SQL dump file (hms.sql) initializes the core database structure and includes seed data for testing. Key tables include:

- users: Stores login credentials and role-based access (admin, doctor, patient)
- doctors: Doctor profiles, specialization, and availability
- appointments: Patient booking data linked to doctors
- posts: Blog/news content posted by admin
- tblmedicalhistory: Medical records and history entries added by doctors
-

The SQL file is typically found in the root or database/ folder and should be imported using phpMyAdmin, MySQL CLI, or deployment scripts.



14. Contact

For queries, feedback, or contributions related to the CARE Hospital Management System project, please reach out to the project lead & team members:

Fatima Nahid

& Team Members

Project: CARE HMS

<https://github.com/fatima-897/CARE.git>

ACKNOWLEDGMENTS

Special thanks to:

- **Mentors & Teachers** – For their guidance and support throughout the development process.
- **Open Source Community** – For the tools, libraries, and inspiration behind many of the components used.
- **Testers** – Whose feedback helped refine the system.

Credits:

This system was designed, developed, and documented by:

Fatima Nahid

Javeria

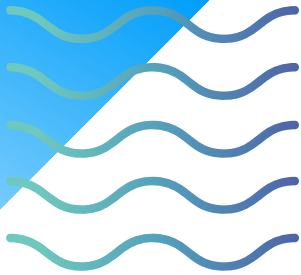
Abdul Ghani

Huzaifa Fahim

Ashar Hussain



CARE



Version Info

Version: 1.0

Last Updated: July 2025

Created By

Fatima Nahid
& Team Members
Project: CARE HMS

Supervisor

Sir Toufique



<https://github.com/fatima-897/CARE.git>

Thank You

We appreciate your trust in CARE — a complete hospital management solution designed to simplify appointments, enhance doctor-patient interaction, and digitize hospital operations.

Need Help? We're Here.

